
Redes neuronales recurrentes

PID_00290600

Toni Lozano Bagén

Tiempo mínimo de dedicación recomendado: 3 horas



Universitat
Oberta
de Catalunya

Toni Lozano Bagén

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

Cómo citar este recurso de aprendizaje con el estilo Harvard:

Lozano Bagén, T. (2023) *Redes neuronales recurrentes*. [Recurso de aprendizaje textual]. 1ª edición. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024

© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoría: Toni Lozano Bagén

Producción: FUOC

Todos los derechos reservados

Ninguna parte de esta publicación, incluyendo el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma ni por ningún medio, sea este eléctrico, químico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita de los titulares de los derechos.

Índice

Introducción	4
Objetivos	5
1. Fundamentos de las redes recurrentes	6
1.1. Concepto de recurrencia	6
1.2. Tipos de redes neuronales recurrentes	10
1.3. Entrenamiento de una red neuronal recurrente	12
2. Tipología de celdas recurrentes	14
2.1. <i>Long Short Term Memory</i> (LSTM)	14
2.1.1. Estructura de las celdas	14
2.1.2. Consideraciones sobre las celdas LSTM	17
2.2. <i>Gated Recurrent Unit</i> (GRU)	18
2.2.1. Estructura de las celdas	18
2.2.2. Consideraciones sobre las celdas GRU	20
3. Arquitecturas de redes recurrentes	21
3.1. Redes neuronales recurrentes bidireccionales	21
3.2. Redes neuronales recurrentes profundas	22
3.3. Arquitectura codificador-decodificador	24
3.4. Mecanismo de atención	25
4. Consejos prácticos y ejemplos	28
4.1. Consejos prácticos en el uso de RNN	28
4.2. Ejemplos	31
Resumen	36
Glosario	37
Bibliografía	38

Introducción

Cuando un conjunto de datos tiene una estructura intrínseca secuencial, como los textos o las series temporales, existe una correlación espacial entre los elementos de cada registro en el conjunto de datos. En estos casos podemos definir las conexiones en una red neuronal de forma que el entrenamiento de la red aproveche al máximo la estructura secuencial de los datos. El resultado son las redes neuronales recurrentes.

Una red neuronal recurrente se puede construir de varias formas. En este módulo didáctico veremos las formas más habituales de construir redes neuronales recurrentes y algunos casos de uso.

Concretamente, en el primer capítulo veremos el concepto fundamental de las redes neuronales recurrentes, la recurrencia, junto con los tipos de redes neuronales recurrentes que se pueden considerar según las características del problema que se quiere resolver. Veremos también cómo se puede utilizar el algoritmo de retropropagación para entrenar una red neuronal recurrente.

En el segundo capítulo analizaremos en detalle la estructura interna de los dos tipos de celdas más comúnmente utilizadas en una red neuronal recurrente: las celdas LSTM (*Long Short Term Memory*) y las celdas GRU (*Gated Recurrent Memory*).

El tercer capítulo está dedicado a considerar las diferentes formas en las que se pueden conectar las celdas de una red neuronal recurrente para conseguir modelos con mayor capacidad predictiva. Se introducirá también en este capítulo el concepto de atención, que es la base de la arquitectura de transformadores que veremos en futuros módulos didácticos.

Por último, en el cuarto capítulo se verá como ejemplo el uso de una red neuronal recurrente para la predicción de una serie temporal generada sintéticamente.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Entender el caso de uso de las redes neuronales recurrentes.
2. Comprender los principales tipos de operaciones que realiza una red neuronal recurrente.
3. Conocer las diferentes arquitecturas que se pueden utilizar con redes neuronales recurrentes.

1. Fundamentos de las redes recurrentes

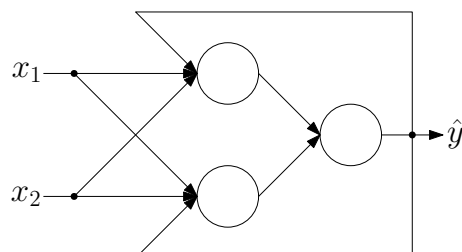
En este capítulo introduciremos los principios y conceptos fundamentales de las redes neuronales recurrentes. En concreto, iniciaremos este capítulo revisando el concepto de «recurrencia» que se aplica en este tipo de redes y, a continuación, veremos los principales tipos de redes neuronales recurrentes. Finalizaremos el capítulo discutiendo el método de entrenamiento en el caso de las redes recurrentes.

1.1. Concepto de recurrencia

Hasta ahora, en todas las redes neuronales que hemos visto, la información fluye siempre en una única dirección: desde las neuronas de entrada hacia las neuronas de salida. Este tipo de redes neuronales se conocen en inglés con el término *feedforward*, pero, en general, en una red neuronal puede haber conexiones que vayan desde una capa posterior hacia una capa anterior.

En la figura 1 podemos ver un ejemplo en el que la capa de neuronas ocultas está conectada a los datos de entrada y, a su vez, a la salida de las neuronas de la propia red neuronal. De esta forma, cuando la red procesa un registro concreto está utilizando, a la vez, información del propio registro e información que generó la red con el registro anterior. Esto crea un estado interno en la red que le permite variar la respuesta a un cierto registro de entrada dependiendo de los registros que ha procesado la red anteriormente. Como veremos más adelante, en muchos casos al estado de la red también se le denomina **memoria** de la red.

Figura 1. Ejemplo de red neuronal recurrente



Este tipo de conexiones pueden ser muy útiles para tratar secuencias de datos, como pueden ser las series temporales o los textos. Por ejemplo, si queremos hacer análisis de sentimiento de una frase, podemos introducir las palabras de la frase una a una en la red neuronal, de forma que en cada paso de la red

el estado va cambiando hasta que introducimos la última palabra de la frase, momento en el cual observamos la salida de la red para determinar si la frase tiene sentimiento positivo o negativo.

Observemos que, en principio, nada impide tratar secuencias de datos con una red completamente conectada como las que hemos visto en módulos anteriores. Para ello deberíamos fijar *a priori* la longitud de las secuencias a tratar e iríamos introduciendo el primer valor de la secuencia en la primera neurona, el segundo valor en la segunda neurona, etc. Uno de los mayores problemas con esta solución aparece cuando existen secuencias que tienen los mismos valores en distinto orden pero aun así tienen el mismo significado. Un ejemplo de esto serían las frases «La película que vi ayer no me gustó mucho» y «No me gustó mucho la película que vi ayer». En una red completamente conectada, las neuronas tienen un orden fijo, por lo que reaccionan a los datos de entrada en un orden concreto. Esto quiere decir que, en este tipo de problemas donde el orden de las palabras no es lo más importante, deberíamos entrenar la red neuronal completamente conectada utilizando todas las variaciones posibles de cada frase, lo cual, en la mayoría de los casos, es inviable.

En los casos donde el orden de las palabras sí que influye en el significado de la frase también es más interesante utilizar redes neuronales recurrentes en lugar de redes neuronales completamente conectadas. La razón es que las neuronas en una red neuronal completamente conectada son «independientes entre sí», por lo que es difícil que una red neuronal completamente conectada sea capaz de aprender que el significado depende del orden de las palabras. En una red neuronal recurrente, en cambio, las palabras se procesan en orden, por lo que estamos forzando a la red a que considere el orden como una fuente de información importante.

En general, las conexiones recurrentes en una red neuronal pueden ser de muchos tipos. Por esta razón, y para poder hacer un estudio unificado de las redes neuronales recurrentes, se define el concepto de «celda» (en inglés, *cell*). Una **celda** es simplemente una operación con dos entradas y dos salidas que se va componiendo consigo misma a medida que se aplica la operación a los diferentes valores de la secuencia. Los puntos de entrada de la celda corresponden, por un lado, a los valores de la secuencia para cada registro y, por otro lado, al estado de la red neuronal en el paso anterior. Los puntos de salida suelen ser el estado de la red en el paso concreto en el que se ejecuta la celda y la respuesta de la red para ese paso.

Como veremos más adelante, en algunos casos las entradas y salidas de una celda pueden bifurcarse o concatenarse para simular tener más o menos de dos entradas y dos salidas, pero el concepto fundamental de celda sigue siendo el mismo.

Formalmente, supongamos que denotamos el paso concreto de una secuencia con la letra t . Si los valores de la secuencia son vectores de dimensión m tenemos que, en el tiempo t , podemos denotar el vector de entrada a la red como $x_t \in \mathbb{R}^m$. De la misma forma, la salida en cada paso de la recurrencia puede ser un vector de dimensión l , por lo que tendremos $h_t \in \mathbb{R}^l$. Por último, debemos fijar un valor n para la dimensión del vector que representa el estado de la red, de forma que tendremos $s_t \in \mathbb{R}^n$. Con estas consideraciones, una celda es una operación f como la siguiente:

$$\begin{aligned} f : \quad \mathbb{R}^{m+n} &\rightarrow \mathbb{R}^{n+l} \\ (x_t, s_{t-1}) &\mapsto (h_t, s_t) \end{aligned} \quad (1)$$

La operación que calcula la celda, f , se suele representar internamente con neuronas y conexiones no recurrentes entre las neuronas. En la figura 2 podemos observar una celda genérica, donde solo mostramos las entradas y salidas, mientras que en la figura 3 podemos visualizar la red de la figura 1 representada con el formato de celda.

Figura 2. Diagrama de una celda, sin especificar las operaciones internas

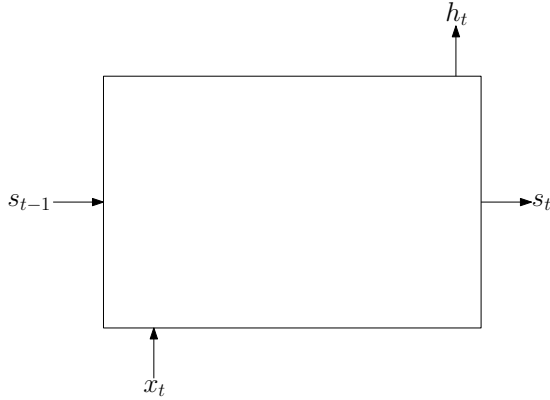
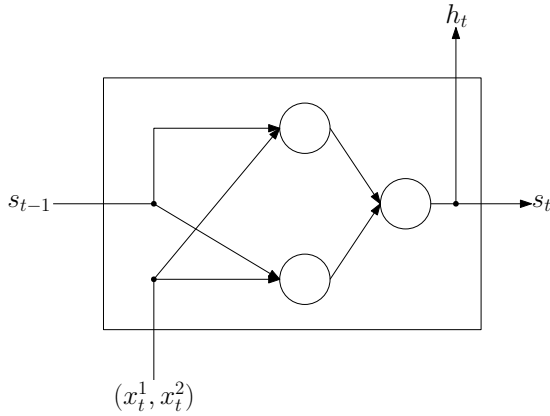


Figura 3. Representación en formato celda de la red recurrente de la figura 1

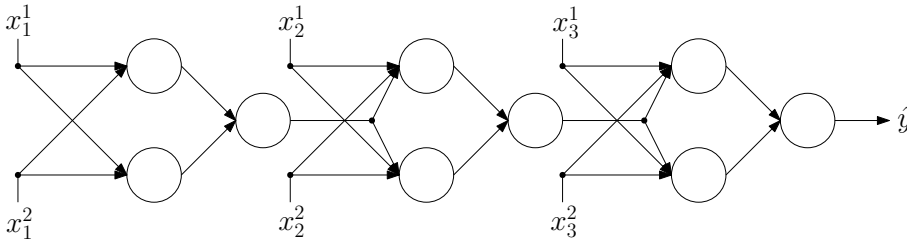


En el ejemplo concreto de la figura 3, cada conexión representa un único valor escalar, pero, en general, las conexiones internas de una celda recurrente pueden trabajar con vectores, de forma que las salidas de la celda están formadas

también por vectores, como se muestra en la ecuación 1. En este caso, normalmente se añaden una o varias capas completamente conectadas a la salida de la última celda recurrente para adaptar el tamaño de la salida al conjunto de datos con el que trabajamos.

Si consideramos una red neuronal recurrente en su máxima generalidad, la salida de la red en un momento dado puede depender de datos tan antiguos como larga sea la secuencia. En la práctica, para hacer más eficiente el uso de las redes neuronales recurrentes, se escoge un valor k que fija la longitud de las secuencias que tratará la red. Si las secuencias a tratar tienen longitud menor que k se añaden valores especiales de relleno (al principio o al final de la secuencia). En cambio, si las secuencias son más largas del valor escogido se cortan y se descarta parte de la secuencia (se debe escoger también si descartar el principio o el final de la secuencia). Una vez fijado el valor de k podemos «desenrollar» la red para convertir la red neuronal recurrente en una red neuronal *feedforward*, como podemos ver en la figura 4.

Figura 4. Red neuronal recurrente desenrollada con $k = 3$



Es importante notar que los parámetros internos de las celdas son compartidos en todos los pasos, por lo que los valores de los parámetros son iguales en cada paso. La diferencia entre un paso y el siguiente estará únicamente en las entradas a la celda que, como hemos dicho, se corresponden con el estado de la red y los datos de entrada. Concretamente, los parámetros internos de la red fijan la función $f : \mathbb{R}^{m+n} \rightarrow \mathbb{R}^{n+l}$, de forma que, si fijamos el valor inicial para s_0 , podemos escribir la recurrencia como:

$$(h_1, s_1) = f(x_1, s_0)$$

$$(h_2, s_2) = f(x_2, s_1)$$

$$\vdots$$

$$(h_t, s_t) = f(x_t, s_{t-1})$$

1.2. Tipos de redes neuronales recurrentes

Como hemos comentado, las redes neuronales recurrentes son especialmente útiles para tratar con secuencias de datos, pero aun así podemos distinguir entre distintos tipos. Dependiendo del problema en el que estemos trabajando, puede interesarnos considerar únicamente algunas de las entradas o salidas de las celdas de la red, lo que da lugar a los siguientes tipos de redes recurrentes:

- Si las secuencias que tratamos tienen varios datos de entrada pero solo nos interesa un único valor que observamos al final de la recurrencia, obtenemos un diagrama como el de la figura 5. Un ejemplo de este caso de uso sería hacer un análisis de sentimiento sobre textos.
- Si trabajamos con secuencias que contienen un único valor pero nos interesa generar una salida con varios pasos, podemos introducir un valor nulo a partir del segundo paso de la recurrencia e ir observando la salida de la red, como aparece en la figura 6. Este tipo de red recurrente se puede utilizar por ejemplo para obtener descripciones dada una única imagen de entrada.
- En el caso en que tratemos secuencias con varios valores y estemos interesados en los valores que genera la red neuronal una vez tratada la secuencia de entrada, podemos proceder como en el caso anterior introduciendo valores nulos una vez procesada la secuencia y observando la salida de la red, como podemos ver en la figura 7. **Si trabajamos con series temporales podemos utilizar este tipo de red recurrente para introducir valores pasados a la red y obtener la predicción de varios valores a futuro.**
- Es posible que estemos interesados en trabajar con secuencias que tienen varios valores y necesitemos obtener una respuesta sincronizada para cada uno de los valores de entrada, en este caso tendríamos un diagrama como el de la figura 8. Un caso de uso de este tipo de red se da, por ejemplo, cuando se trabaja con un vídeo fotograma a fotograma y se desean clasificar objetos en cada fotograma.

Figura 5. Red neuronal recurrente con varios datos de entrada y una única salida

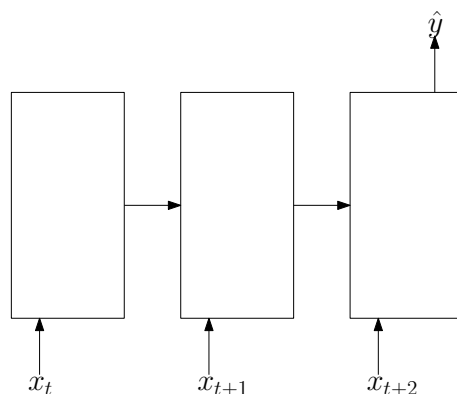


Figura 6. Red neuronal recurrente con un único dato de entrada y varios datos de salida

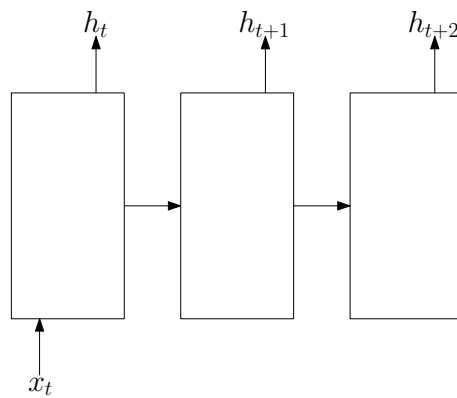


Figura 7. Red neuronal recurrente con varios datos de entrada y varios datos de salida

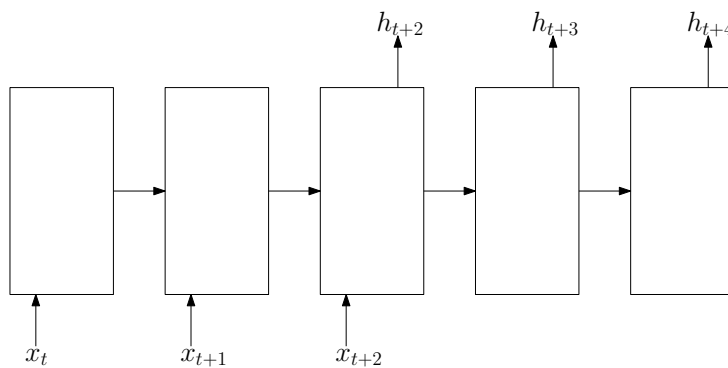
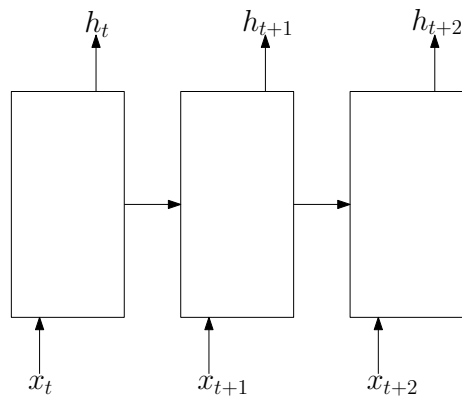


Figura 8. Red neuronal recurrente con varios datos de entrada y varios datos de salida sincronizados



Es importante observar que todas las celdas tienen siempre dos entradas y dos salidas. En cada una de las figuras 5, 6, 7 y 8, si no aparece alguna de las salidas es porque esa salida en concreto se ignora, mientras que si no aparece alguna de las entradas es porque en esos puntos se introduce un valor concreto que actúa como valor nulo.

1.3. Entrenamiento de una red neuronal recurrente

En el módulo sobre redes neuronales completamente conectadas hemos visto el algoritmo de retropropagación (*backpropagation*), que permite calcular de forma eficiente el gradiente de una función de coste para una red neuronal *feedforward*. En el caso de una red neuronal recurrente no se puede aplicar directamente el algoritmo de retropropagación debido a las conexiones entre neuronas de capas posteriores con neuronas de capas anteriores.

En general, hemos visto que entrenar una red neuronal es encontrar unos valores para los parámetros de la red que minimicen una determinada función de coste, y esto es igual en el caso de las redes recurrentes. Por esta razón, se pueden entrenar las redes neuronales recurrentes con cualquier algoritmo de optimización, como por ejemplo los algoritmos genéticos. Aun así, dado el alto rendimiento del algoritmo de retropropagación para otros tipos de redes neuronales, es lógico pensar que una adaptación del mismo algoritmo puede resultar útil también en el caso de las redes recurrentes.

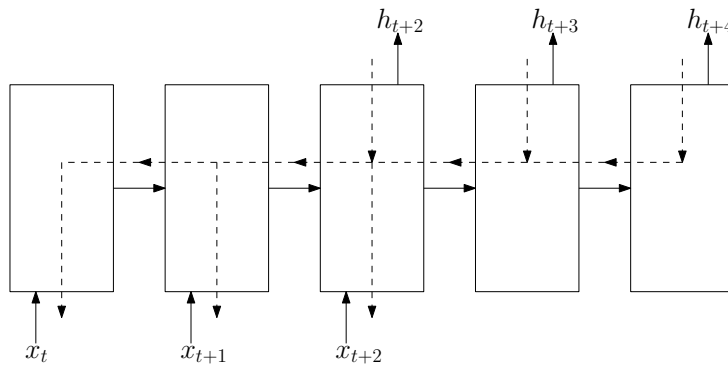
Esta adaptación se conoce con el nombre de retropropagación en el tiempo (en inglés, *backpropagation through time*) y no es más que desenrollar la red recurrente para convertirla en una red *feedforward* y aplicar el algoritmo de retropropagación. Hay que tener en cuenta, eso sí, el tipo de red que estemos utilizando ya que, según hemos visto en la sección anterior, en ocasiones nos puede interesar trabajar con la salida de la red en diferentes pasos, lo cual afecta directamente a la función de coste. Recordemos que en una red recurrente todos los parámetros internos de la celda son compartidos en todos los pasos, por lo que el gradiente de la función de coste puede afectar a un mismo parámetro de la red en repetidas ocasiones, dependiendo de en qué pasos se estén considerando las salidas de la red para calcular la función de coste.

Si nos fijamos en la figura 4, podemos intuir que el método de retropropagación calculará el error en cada una de las salidas que estemos considerando y lo propagará hacia atrás en cada uno de los pasos, tal y como se muestra en la figura 9. Esto quiere decir que, si consideramos la salida de la red a cada paso, cada parámetro de la red neuronal contribuirá al gradiente de la función de coste una vez por cada uno de los pasos en los que hayamos desenrollado la red. Cuánto contribuirá exactamente cada parámetro en cada momento dependerá del paso concreto del que se trate y de los valores de entrada de la red.

En una red neuronal en la que haya conexiones recurrentes que vayan directamente de capas posteriores a capas anteriores se puede demostrar, haciendo el cálculo analítico, que el gradiente de la función de coste con respecto a los parámetros en un paso concreto de la red depende del producto del gradiente en los pasos posteriores. Esto implica que, si calculamos cuánto contribuye al gradiente un parámetro concreto en uno de los pasos iniciales de la red, el

gradiente aparecerá elevado a una potencia, que puede ser muy alta si hemos desenrollado la red en un gran número de pasos intentando capturar dependencias lo más lejanas posibles.

Figura 9. Flujo del gradiente de la función de coste en una red recurrente



El hecho de que el gradiente se vea multiplicado consigo mismo en el cálculo de cómo afectan los parámetros en cada paso provoca dos problemas diferentes pero relacionados, que ya hemos comentado anteriormente, pero que volvemos a resumir a continuación.

- **Desaparición del gradiente:** si la norma del gradiente es menor que 1, al multiplicar el gradiente consigo mismo muchas veces el resultado es prácticamente 0, por lo que los primeros pasos de la red apenas aportan nada al cálculo del gradiente. Esto implica que la red no será capaz de aprender dependencias entre valores alejados de una secuencia, ya que el valor de los parámetros de la red se verá influenciado únicamente por los últimos pasos de la recurrencia.
- **Explosión del gradiente:** si la norma del gradiente es mayor que 1, al multiplicar el gradiente consigo mismo muchas veces el resultado es inmensamente grande. Esto provoca que el entrenamiento de la red sea completamente inestable, ya que las modificaciones que se aplican a los parámetros utilizando el método de retropropagación hacen que haya demasiadas fluctuaciones y la red no consigue aprender nada.

2. Tipología de celdas recurrentes

La solución más utilizada para resolver los problemas de explosión y desaparición del gradiente consiste en utilizar celdas en las que existe una conexión entre el estado anterior y la salida de la celda a la cual no se aplican funciones no lineales. En este tipo de celdas se controla la información que fluye por esta conexión directa utilizando un concepto similar a las puertas lógicas, modeladas con neuronas completamente conectadas.

En este capítulo veremos las dos arquitecturas de celdas más utilizadas de este tipo, la arquitectura LSTM y la arquitectura GRU.

2.1. *Long Short Term Memory* (LSTM)

En muchos casos las secuencias que queremos analizar con una red neuronal recurrente pueden presentar dependencias entre puntos de la secuencia muy separados. Por ejemplo, si utilizamos una red neuronal recurrente para analizar un texto es posible que dentro del texto se hagan referencias a conceptos que se han mencionado cientos de palabras antes, por lo que, si nuestro objetivo es que la red sea capaz de extraer información del texto, es posible que necesitemos que la red neuronal trate cientos de pasos.

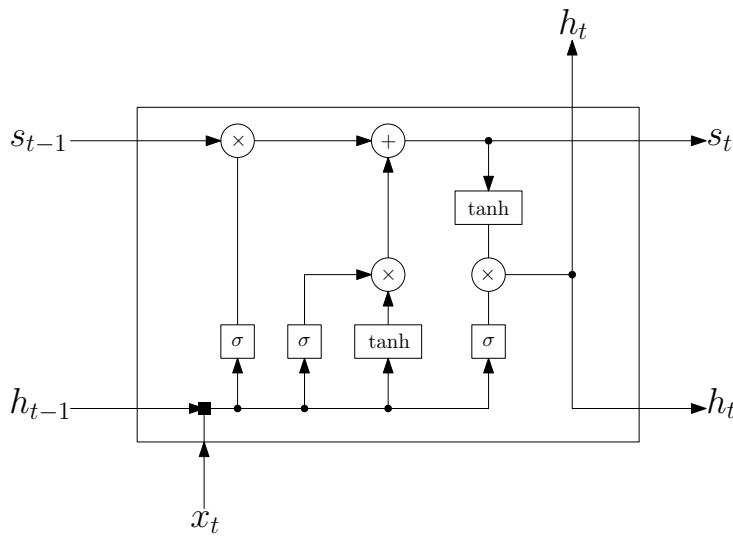
2.1.1. Estructura de las celdas

Este tipo de modelos presentan siempre problemas como la desaparición o explosión del gradiente. Para intentar mitigar estos problemas, en 1997 se propuso el uso de una arquitectura de celda muy concreta llamada *Long Short Term Memory* (LSTM). Las entradas de una celda LSTM son, por una parte, el valor de la secuencia en el paso correspondiente y, por otra parte, la concatenación del estado y la salida de la red en el paso anterior. Normalmente, para evitar tener que concatenar vectores a la salida de un paso para separarlos justo al inicio del paso siguiente se suelen representar las celdas LSTM con tres entradas, como podemos observar en la figura 10.

Lectura complementaria

S. Hochreiter, J. Schmidhuber. «Long Short-Term Memory». *Neural Computation*, Vol. 9 (8), p. 1735-1780, 1997.

Figura 10. Diagrama de una celda LSTM



Una red LSTM puede aprender tanto dependencias entre puntos alejados de una secuencia, como dependencias entre puntos cercanos. Los puntos claves para que la red tenga esta habilidad son dos:

- 1) El **canal de memoria** que se puede observar como una línea horizontal en la parte superior de la celda.
- 2) El **control del flujo de información** mediante puertas.

Como podemos ver en el diagrama, la arquitectura de la celda LSTM se basa en controlar qué información se guarda en el **estado** (también llamado **memoria**) y fluye directamente por la parte superior y qué información se produce como respuesta de la red. Este control se ejerce mediante las siguientes puertas:

Puerta de olvido

La puerta de olvido (*forget gate*) observa el valor de entrada a la celda junto con la salida de la red en el paso anterior y decide qué parte de la memoria se debe conservar. Concretamente, la puerta de olvido calcula un vector f_t , de la misma longitud que los vectores de estado, con la siguiente fórmula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

donde W_f es una matriz de parámetros que multiplica a la concatenación de h_{t-1} y x_t , y b_f es un vector de sesgos.

Para imaginar el uso de la puerta de olvido, supongamos que tenemos un conjunto de datos con el consumo diario de un cierto producto y queremos predecir el consumo durante los próximos 3 días. Asumamos, además, que por

las características del producto el consumo en un día concreto únicamente depende del consumo que se haya producido en la misma semana. En este caso, si en cada paso introducimos a la red el consumo de un día y añadimos información sobre el día de la semana del que se trata, es posible que la puerta de olvido aprenda a borrar toda la memoria cuando la entrada indique que está tratando con el consumo de un lunes. De esta forma, toda la información pasada será eliminada del estado y los pasos siguientes no tendrán información sobre el consumo la semana anterior, lo cual puede ser útil en un problema como el descrito.

Puerta de entrada

La puerta de entrada controla qué información se añade a la memoria de la red. Es una puerta completamente análoga a la puerta de olvido, pero no comparten parámetros. La fórmula concreta que calcula la puerta de entrada es la siguiente:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

donde W_i es una matriz de parámetros y b_i es un vector de sesgos.

Como caso de uso simple, imaginemos que estamos trabajando con diferentes frases y queremos extraer si el objeto principal de la frase es masculino o femenino. En este caso, habrá palabras que den información relevante y otras que no, por lo que es posible que la red neuronal aprenda a reconocer las palabras que aportan información y deje pasar únicamente la información que corresponda a las palabras útiles para el problema en cuestión.

A la vez que se calculan los valores de la puerta de entrada y, utilizando exactamente la misma información pero con diferentes parámetros, se calcula qué información concretamente es candidata a añadirse a la memoria con la siguiente fórmula:

$$\tilde{s}_t = \tanh(W_s \cdot [h_{t-1}, x_t] + b_s) \quad (4)$$

donde W_s vuelve a ser otra matriz diferente de parámetros y b_s es otro vector diferente de sesgos.

Por último, una vez se ha calculado la información candidata a añadirse a la red y los valores de la puerta de entrada y salida, el estado de la red se actualiza de la siguiente forma:

$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \quad (5)$$

donde $\odot$ denota el producto de vectores componente a componente.

Como se puede apreciar en la última fórmula, el nuevo estado de la red viene dado por una suma ponderada entre el estado anterior y la nueva información, donde los pesos de cada componente vienen dados por la puerta de olvido y la puerta de entrada, respectivamente.

Puerta de salida

Una vez se ha calculado el nuevo estado que tendrá la red hay que decidir cuál será la salida de la red en el paso en el que nos encontramos. Para eso utilizamos la puerta de salida, cuyo proceso es similar al de la puerta de entrada. Concretamente, la puerta de salida observa también el valor de entrada a la celda junto con la salida anterior de la red y calcula un vector o_t de la siguiente forma:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6)$$

Este vector marcará qué información se da como respuesta, la cual vendrá dada por la siguiente fórmula:

$$h_t = o_t \odot \tanh(s_t) \quad (7)$$

Como podemos observar, la salida de la red consistirá en una transformación no lineal del estado actual de la red, modulada según el valor de la puerta de salida.

Enlazando con los ejemplos anteriores, es intuitivo pensar que la memoria de la red contendrá información útil para dar una respuesta en el paso concreto en el que se encuentra la red, pero también contendrá información útil para los pasos siguientes, por lo que la puerta de salida se encarga de dejar pasar únicamente la información que resulta útil en el momento actual.

2.1.2. Consideraciones sobre las celdas LSTM

Observemos que todas las fórmulas anteriores describen exactamente diferentes capas de una red neuronal, ya sea con la función sigmoide o con la tangente hiperbólica como funciones de activación. Esta forma de construir las celdas es lo que da el nombre de redes neuronales recurrentes.

El punto más interesante de controlar una memoria con puertas, como en el caso de una LSTM, es que el control se hace de forma difusa, ya que las puertas pueden tomar cualquier valor real entre 0 y 1, de forma que pueden dejar pasar información solo en parte.

Por ejemplo, en el caso de intentar predecir el consumo de un producto, imaginemos que la dependencia no es únicamente con los días de la misma semana, pero sí sabemos que la tendencia en el consumo disminuye un 20 % los domingos. En este caso concreto, la red neuronal podría aprender a mantener el valor de la tendencia en alguna componente del vector de estado y, de esta forma, cuando la entrada a una celda indique que se está considerando un domingo, es posible que la puerta de olvido multiplique por 0.8 la componente concreta de la memoria que guarda la tendencia del consumo.

De forma parecida, en el ejemplo en el que queremos extraer información de una frase, es posible que no haya una única palabra que contenga toda la información, si no que para lograr el objetivo del modelo se necesite información de varias palabras. En este caso, la puerta de entrada de la red podría aprender a detectar cuánta información relevante contiene cada palabra y modular el paso de información hacia la memoria de la red.

Anteriormente vimos que las redes neuronales recurrentes simples no son capaces de modelar largas dependencias por culpa de los problemas de desaparición y explosión del gradiente. En el caso de las LSTM se puede comprobar que, gracias a las fórmulas utilizadas para calcular el estado, la información del gradiente de la función de coste puede fluir hacia atrás sin multiplicarse consigo misma un número elevado de veces. Este hecho permite que valores de entrada en los primeros pasos puedan tener un efecto importante en el entrenamiento de la red y aprender así dependencias entre puntos alejados de una secuencia.

Otra técnica muy utilizada que ayuda a evitar el problema de la explosión del gradiente es el recorte del gradiente (en inglés, *gradient clipping*). El **recorte del gradiente** consiste en fijar un valor máximo para cada componente del gradiente, de forma que si alguna componente del gradiente es superior al valor fijado se le asigna directamente el valor máximo.

2.2. Gated Recurrent Unit (GRU)

Dado el alto rendimiento de las redes LSTM se han investigado diferentes variantes de celdas en las que se controla el flujo de información utilizando capas de neuronas como puertas que dejan pasar más o menos información hacia la memoria de la red. Una de las variantes que se ha demostrado más útil se introdujo en 2014 con el nombre de *Gated Recurrent Unit* (GRU).

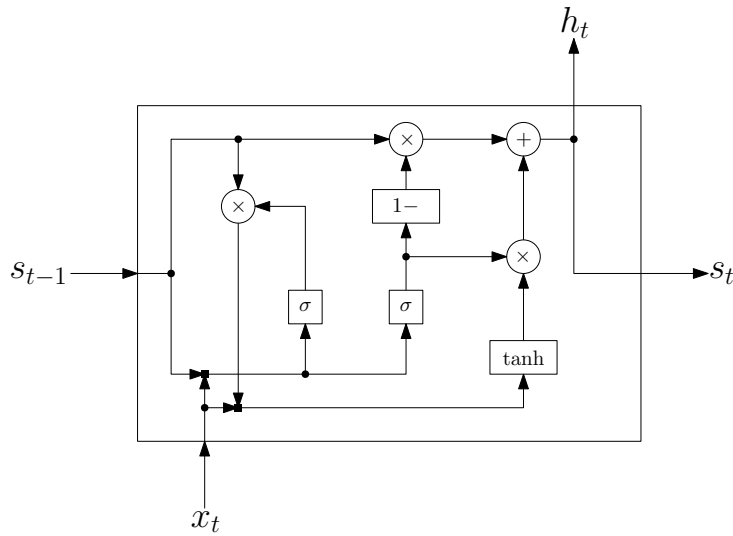
2.2.1. Estructura de las celdas

En la figura 11 podemos ver el diagrama de una celda GRU, en la cual podemos observar que las dos salidas que normalmente hay en una celda son exactamente iguales.

Lectura complementaria

K. Cho *et al.* «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation». Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), p. 1724-1734, 2014.

Figura 11. Diagrama de una celda GRU



Una celda GRU se puede ver como una simplificación de una celda LSTM, en la que se fusiona el estado con la respuesta de la red y, además, se fuerza a que el estado de la red sea una media ponderada entre el estado anterior y la nueva información, pero también existen ligeras diferencias en el cálculo concreto que se realiza para obtener la respuesta de la red.

Las redes recurrentes con celdas GRU han demostrado en varios casos dar rendimientos muy similares a las redes recurrentes con celdas LSTM. Por esta razón, las celdas GRU son muy utilizadas cuando el conjunto de datos no es extremadamente grande, ya que en una celda GRU hay menos parámetros y por lo tanto se necesitan menos datos para el entrenamiento.

Concretamente, en una celda GRU el flujo de información se controla con las puertas de *reset* y las puertas de actualización, que pasamos a describir a continuación.

Puerta de *reset*

La puerta de *reset* permite seleccionar qué información de la memoria va a ser utilizada en un paso concreto. Para ello, observa el estado anterior de la red y los datos de entrada y obtiene un vector que será multiplicado posteriormente con el estado de la memoria. En este caso concreto la fórmula utilizada es la siguiente:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (8)$$

donde W_r es una matriz de parámetros que multiplica a la concatenación de h_{t-1} y x_t , y b_r es un vector de sesgos.

Puerta de actualización

La puerta de actualización es análoga a las puertas de olvido y entrada de una celda LSTM. En particular, esta celda controla qué información se mantiene en la memoria y qué información de la memoria se olvida. En primer lugar se calcula un vector de la misma longitud que el vector de estado con la siguiente fórmula:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (9)$$

donde W_z es una matriz de parámetros y b_z es un vector de sesgos.

A continuación se calcula la información candidata a añadirse a la memoria (recordemos que en el caso de una celda GRU el vector de estado de la red y el vector de salida coinciden). En una celda GRU este cálculo utiliza los datos de entrada, el estado anterior de la red y la salida de la puerta de *reset*, tal y como muestra la fórmula siguiente:

$$\tilde{s}_t = \tanh(W_s \cdot [r_t \odot h_{t-1}, x_t] + b_s) \quad (10)$$

donde W_s es una matriz de parámetros y b_s es un vector de sesgos.

Por último, se actualiza el estado de la red utilizando una media ponderada entre el estado anterior y la información recién calculada:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{s}_t \quad (11)$$

2.2.2. Consideraciones sobre las celdas GRU

Como hemos comentado anteriormente, una celda GRU es, en cierto modo, una simplificación de una celda LSTM, pero aun así existen otras variantes de celdas que simplifican aún más la arquitectura de la celda para necesitar el mínimo número de parámetros posible y poder así entrenar la red recurrente con un conjunto de datos relativamente pequeño.

3. Arquitecturas de redes recurrentes

Una vez sabemos qué tipo de celda queremos utilizar necesitamos decidir cómo se van a organizar las celdas. En este capítulo veremos las técnicas más comúnmente utilizadas para conectar celdas recurrentes de forma que podamos obtener el máximo rendimiento dependiendo del problema que queramos resolver.

3.1. Redes neuronales recurrentes bidireccionales

En muchos casos de uso de una red recurrente la secuencia de datos tiene una clara componente temporal, como por ejemplo en las series temporales. En otros casos de uso, sin embargo, los datos de entrenamiento están formados por secuencias que no tienen una componente temporal subyacente, como por ejemplo en el caso de los textos.

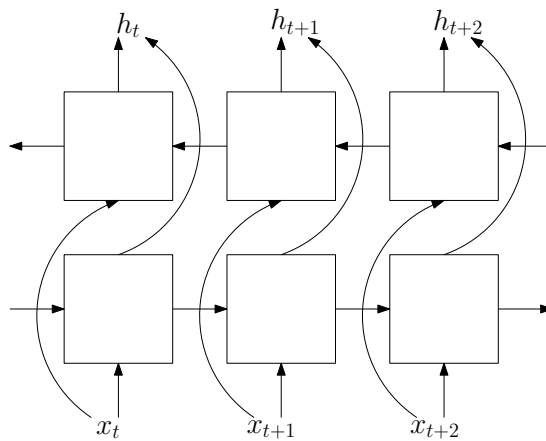
En estos casos en los que trabajamos con secuencias aisladas sin componente temporal es posible introducir los valores de entrada a la red, tanto en el orden original de los datos como en el orden inverso. Un ejemplo claro de que es importante el orden en el que se introducen los datos en la red se da cuando se trabaja con textos. En efecto, dependiendo del idioma de un texto, las palabras más relevantes pueden estar al principio o al final de la frase. De la misma forma, las construcciones en las que ciertas palabras se ven modificadas por palabras adyacentes pueden tener diferente orden en diferentes idiomas.

En general, en estos casos sin componente temporal subyacente, para obtener el máximo posible de información es posible utilizar **redes neuronales recurrentes bidireccionales**. En este tipo de redes primero se aplica una recurrencia en un orden y luego se aplica otra recurrencia en el orden inverso, para después aglutinar toda la información recogida en una capa final. La figura 12 muestra un diagrama de una red neuronal recurrente bidireccional, en la que se puede utilizar una capa completamente conectada para aglutinar la información sobre los estados de las celdas recurrentes.

Lectura complementaria

M. Schuster, K. K. Paliwal.
«Bidirectional Recurrent
Neural Networks». Trans.
Sig. Proc., Vol. 45 (11),
p. 2673-2681, 1997.

Figura 12. Diagrama de una red neuronal recurrente bidireccional



Es importante observar en la figura 12 cuáles son las conexiones entre las diferentes partes de la arquitectura de la red. En particular, podemos ver que las dos recurrencias de la red son independientes, recibiendo cada parte de la recurrencia el estado anterior respectivo y los valores de entrada (en orden diferente). La información que proporciona cada celda en cada uno de los pasos se concatena una vez finalizado el análisis de la secuencia, tanto en un orden como en orden inverso, y se utiliza una capa completamente conectada para extraer la información útil según el problema. En la sección 3.4 veremos una alternativa a la concatenación directa de todas las salidas de las celdas.

En una red neuronal recurrente bidireccional las celdas que van en una misma dirección comparten parámetros, pero los parámetros de las celdas que van hacia una dirección son diferentes de los parámetros de las celdas que van en dirección contraria. De esta forma, el número de parámetros en una red neuronal recurrente bidireccional es el doble de lo que sería en una red neuronal equivalente que no fuera bidireccional.

Este tipo de arquitecturas son muy útiles, por ejemplo, en problemas de clasificación de texto. En este caso podemos entender la red neuronal como dos agentes que extraen el máximo posible de información del texto, uno leyendo el texto en el orden original y el otro leyendo el texto en el orden inverso, para luego compartir la información y hacer una clasificación definitiva utilizando una capa completamente conectada.

3.2. Redes neuronales recurrentes profundas

Si observamos la arquitectura de una celda LSTM o una celda GRU, en las figuras 10 y 11, podemos ver que las operaciones que se realizan sobre los datos de entrada son como si tuviéramos una red neuronal con una única capa. Es cierto que existe un cierto grado de composición entre las operaciones dada la recurrencia de la red, pero esta composición se hace siempre utilizando los

mismos parámetros y cambiando únicamente los datos de entrada en cada paso. Esto, como en el caso de las redes neuronales completamente conectadas, provoca que la red no tenga una gran capacidad de abstracción.

Para intentar dotar de más flexibilidad a una red recurrente podemos proceder como en el caso de las redes completamente conectadas y apilar capas de celdas recurrentes unas encima de otras. En la figura 13 podemos ver exactamente cómo podemos conectar las entradas y salidas de varias celdas para crear una **red neuronal recurrente profunda**.

Figura 13. Ejemplo de una red neuronal recurrente profunda desenrollada

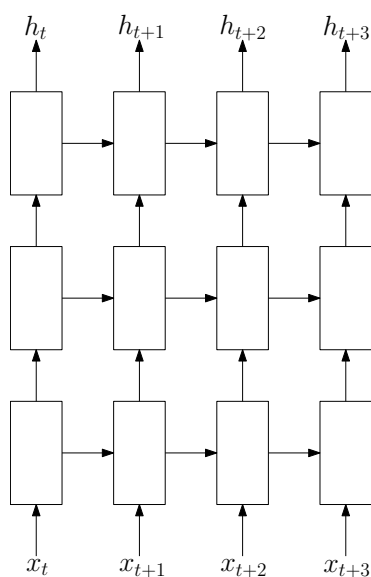
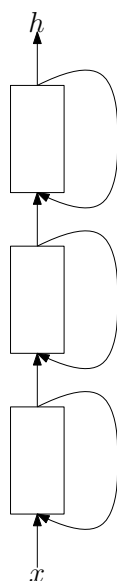


Figura 14. Red neuronal recurrente de la figura 13 sin desenrollar



Es importante tener en cuenta qué parámetros son compartidos entre las celdas de la red. En la figura 14 podemos ver la misma red recurrente que aparece desenrollada en la figura 13. Como podemos ver, las celdas recurrentes de una

Lectura complementaria

A. Graves *et al.* «Speech recognition with deep recurrent neural networks». In International Conference on Acoustics, Speech and Signal Processing, p. 6645-6649, 2013.

misma capa comparten parámetros, pero cada una de las capas es independiente, por lo que no comparten parámetros.

Si observamos de nuevo la figura 13, podemos apreciar que, a diferencia de lo que ocurre en una red recurrente como la de la figura 4, la información que entra a la red pasa por diferentes capas con diferentes parámetros. Esto permite a la red especializar cada capa en conceptos ligeramente más complejos que los que trata la capa anterior, por lo que, de la misma forma que pasa con las CNN, la red es capaz de aprender conceptos muy abstractos. Pero, como en todas las redes neuronales, este aumento de la capacidad de abstracción viene asociado a una mayor complejidad del proceso de entrenamiento.

En una red neuronal recurrente profunda se unen el problema de la desaparición y la explosión del gradiente a la complejidad de entrenar una red neuronal con varias capas. Es completamente indispensable disponer de un conjunto de datos muy grande para poder entrenar una red neuronal recurrente profunda.

Además, es muy importante ajustar correctamente los diferentes hiperparámetros del entrenamiento para obtener una red neuronal estable. Esto puede provocar que debamos probar diferentes combinaciones de hiperparámetros buscando la estabilidad, lo que, junto con el tiempo de entrenamiento de la red, puede alargar mucho el tiempo de desarrollo de una red de este tipo.

3.3. Arquitectura codificador-decodificador

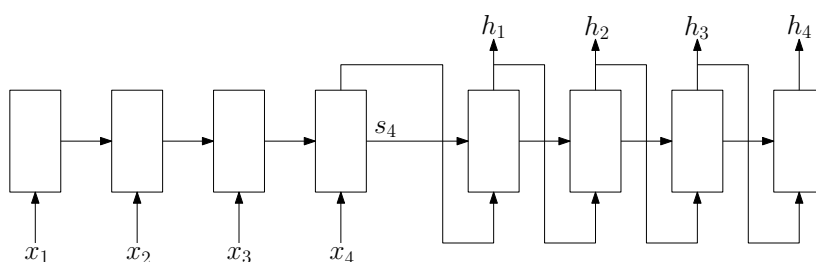
En la figura 7 hemos visto que en una red neuronal recurrente podemos tener, a la vez, secuencias de entrada y secuencias de salida. Un caso particular de esta combinación de entradas y salidas lo podemos ver en la figura 15, en la que primero la red neuronal consume completamente la secuencia de entrada y, posteriormente, genera la secuencia de respuesta.

En una red neuronal con este tipo de arquitectura se denomina **codificador** a la primera parte de la red y **decodificador** a la segunda parte.

Lectura complementaria

I. Sutskever *et al.* «Sequence to Sequence Learning with Neural Networks». Proceedings of the 27th International Conference on Neural Information Processing Systems, Vol. 2, p. 3104-3112, 2014.

Figura 15. Red neuronal recurrente con arquitectura *encoder-decoder* / *sequence-to-sequence*



La información que pasa del codificador al decodificador está toda comprimida en el **vector de estado** del último paso del codificador. En este sentido, el trabajo del codificador consiste en guardar toda la información de la secuencia de entrada que sea relevante para el problema que estamos tratando en un vector que luego el decodificador sea capaz de tratar.

Por otra parte, el trabajo del decodificador consiste en crear una secuencia de salida basada únicamente en la información que recibe del codificador.

El caso de uso típico de este tipo de arquitectura se da en los problemas de traducción automática. En estos casos, la red primero consume toda la secuencia de entrada para obtener toda la información de una frase y después genera la frase traducida palabra a palabra. **El trabajo del codificador en este caso consiste en comprimir toda la información contenida en una frase en un vector, de forma que después el decodificador sea capaz de interpretar el contenido del vector y generar una frase en un nuevo idioma que tenga el mismo significado que la frase original.**

Como en muchos otros casos, el punto clave de este tipo de redes neuronales está en conseguir representar la información relevante en un vector numérico. Para obtener unos vectores lo más representativos posible se pueden combinar las técnicas que hemos visto en las secciones anteriores sobre redes bidireccionales y redes recurrentes profundas.

En este sentido, dependiendo del problema, es posible que una red neuronal bidireccional sea capaz de obtener una mejor representación vectorial de la secuencia de entrada, de forma que tener un codificador bidireccional hará que el decodificador pueda obtener mejores resultados.

De la misma forma, si las secuencias de entrada contienen conceptos abstractos, es muy posible que utilizando diversas capas en el codificador y en el decodificador permita obtener mejores resultados. Estas mejoras conllevan, como es de esperar, una mayor complejidad en el entrenamiento de la red neuronal, por lo que normalmente es necesario disponer de una gran cantidad de datos para entrenar una red neuronal con arquitectura codificador-decodificador.

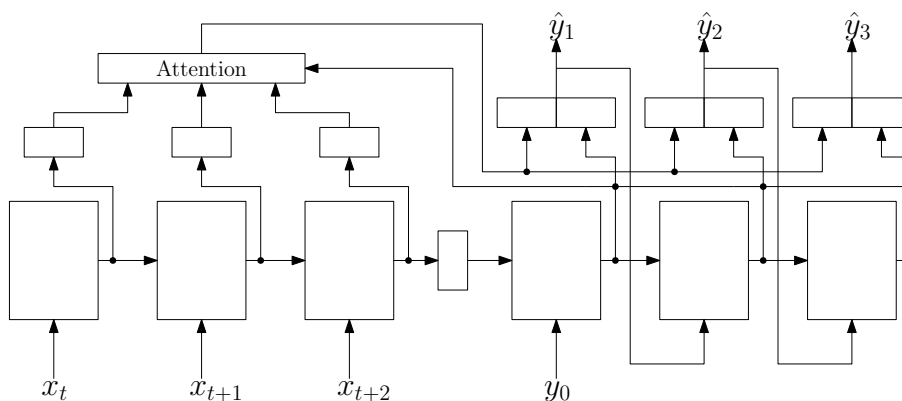
3.4. Mecanismo de atención

Como hemos visto anteriormente, con una arquitectura codificador-decodificador se puede entrenar un modelo que permita traducir frases entre idiomas. En este tipo de arquitecturas, es intuitivo pensar que la parte más compleja del modelo está en sintetizar en un único vector toda la información contenida en una frase, para después poder generar una frase en otro idioma.

Para mejorar en este proceso de compresión de la información, en 2015 se introduce el mecanismo de atención, cuyo objetivo es ayudar a construir un vector que condense la información contenida en una frase de la mejor forma posible.

Concretamente, con el **mecanismo de atención**, en lugar de utilizar el vector de estado del último paso de la red recurrente como representación de toda la secuencia, se utiliza una suma ponderada de los estados de cada paso de la red codificadora. En la figura 16 podemos ver una representación del mecanismo de atención.

Figura 16. Mecanismo de atención en una red neuronal recurrente con arquitectura *encoder-decoder / sequence-to-sequence*



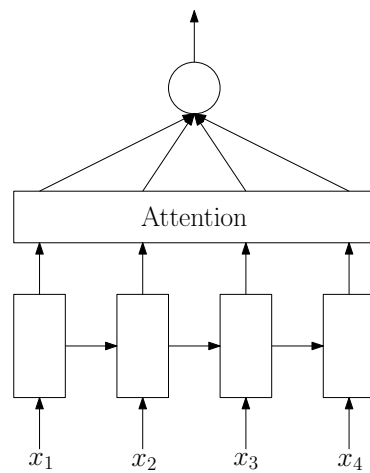
La idea fundamental es que, en la suma ponderada, los pesos que se asocian a cada paso de la entrada dependen del paso concreto de la salida en el que se encuentre el proceso de decodificación. Esta suma ponderada de todos los estados se concatena después con el estado de la red decodificadora para usarse como base en el cálculo de la salida de la red, lo que permite a la red centrar la atención en ciertos puntos de la secuencia en cada momento. De esta forma, es posible representar más fidedignamente las partes importantes de la secuencia en el vector que se utiliza en cada paso del decodificador.

El mecanismo de atención también puede ser utilizado con una red recurrente en un problema de regresión o clasificación habitual. El proceso es exactamente el mismo que con una arquitectura codificador-decodificador, pero en lugar de calcular una suma ponderada de los estados del codificador para cada paso del decodificador, se realiza una única suma ponderada al finalizar de tratar cada secuencia de entrada. En la figura 17 podemos ver un diagrama con una red neuronal con mecanismo de atención aplicado a un problema de clasificación.

Lectura complementaria

D. Bahdanau *et al.* «Neural Machine Translation by Jointly Learning to Align and Translate». 3rd International Conference on Learning Representations, 2015.

Figura 17. Mecanismo de atención en una red neuronal recurrente para un problema de clasificación



Este mecanismo puede ser muy útil en casos en los que una parte de una secuencia es la que lleva toda la información relevante para el problema. Por ejemplo, si queremos extraer el tiempo verbal de una frase es probable que la información relevante se encuentre en una zona muy concreta de la frase que no tiene por qué estar al final. Utilizando un mecanismo de atención, la red puede aprender a detectar los verbos, aislarlos y luego clasificar el tiempo verbal sin prestar atención a otras partes de la frase que pueden aportar ruido.

4. Consejos prácticos y ejemplos

En este último capítulo del módulo de redes neuronales recurrentes nos centraremos en ver algunos consejos prácticos a la hora de trabajar con RNN y series temporales para la creación de modelos predictivos. Finalmente, veremos algunos ejemplos de problemas que se pueden resolver empleando las RNN.

4.1. Consejos prácticos en el uso de RNN

Durante esta parte dedicada a las RNN hemos visto algunos ejemplos de problemas que se pueden resolver con redes neuronales recurrentes. Sin embargo, es importante tener presente que cada problema tiene unas características especiales que requerirán diferentes soluciones, ya sea en la arquitectura de la red neuronal o en el procesamiento de los datos.

En el caso de trabajar con series temporales, por ejemplo, es importante utilizar técnicas estadísticas clásicas para preprocesar los datos, como la normalización o la eliminación de tendencias, para obtener los mejores resultados posibles del entrenamiento.

Una de las técnicas más utilizadas es la descomposición de una serie temporal en tres componentes: tendencia, estacionalidad y componente irregular. Esta descomposición se puede realizar de dos formas diferentes, aditiva y multiplicativa. Si tenemos una serie temporal y_t , aplicando una descomposición aditiva obtenemos

$$y_t = T_t + S_t + I_t$$

mientras que si aplicamos una descomposición multiplicativa obtenemos

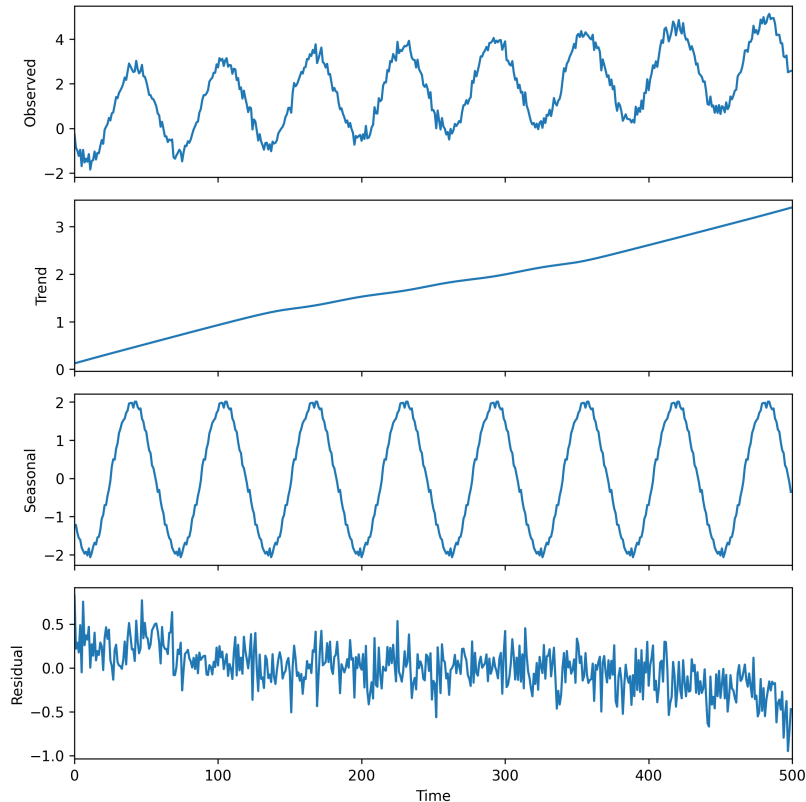
$$y_t = T_t \cdot S_t \cdot I_t$$

donde T_t denota la tendencia, S_t la estacionalidad e I_t la componente irregular. En la figura 18 podemos ver un ejemplo de una serie temporal descompuesta de forma aditiva.

Utilizando la descomposición en tres componentes transformamos el problema de entrenar una red neuronal para predecir una serie temporal a tener que predecir tres series temporales diferentes. La ventaja en este caso es que

las componentes serán más fáciles de predecir, especialmente la parte de la tendencia y de la estacionalidad.

Figura 18. Serie temporal y su descomposición en tendencia, estacionalidad y componente irregular



Posteriormente, cuando queramos predecir un nuevo valor de la serie original, simplemente deberemos predecir los valores de las tres componentes y agregarlos utilizando el mismo método que utilizamos para la descomposición, ya sea aditiva o multiplicativa.

En el caso de que estemos utilizando una red neuronal recurrente para trabajar con textos, es muy importante seleccionar una forma adecuada para representar numéricamente las palabras que forman cada registro.

La forma más simple de representar un texto consiste en fijar primero un diccionario con N palabras (por ejemplo, 1.000, 2.000 o 5.000 palabras). Entonces, para cada texto se construye un vector de N dimensiones para cada una de las palabras. Cada vector tiene todas las componentes igual a cero excepto una componente, que tiene el valor 1, e indica qué palabra del diccionario estamos representando con el vector.

Esta forma de representar textos tiene varios inconvenientes, como por ejemplo:

- Cada palabra está descrita por un vector de N dimensiones, donde N puede ser un valor muy grande si el diccionario contiene muchas palabras. Por lo tanto, no es adecuado, en la mayoría de los casos, porque incrementa de forma considerable el tamaño de la red neuronal, aumentando la complejidad del entrenamiento.
- Las palabras tienen relaciones de similitud que no se ven reflejadas en los vectores. Es decir, los vectores correspondientes a dos palabras que tengan significado parecido tendrán la misma relación que los vectores correspondientes a dos palabras completamente diferentes.

Esto vuelve a incrementar la complejidad del entrenamiento, ya que dos textos que tengan significados similares tendrán representaciones vectoriales sin ninguna relación, por lo que la red tendrá que aprender por separado sobre los dos textos y no podrá utilizar información aprendida con un ejemplo a la hora de tratar con ejemplos relacionados.

Para solucionar estos dos problemas se suele utilizar una capa de *embedding* justo antes de la entrada de la red neuronal.

Esta capa es simplemente una matriz de tamaño $N \times k$, donde N es el número de palabras que tenemos en el diccionario y k es la dimensión del *embedding*, que suele estar entre 50 y 300, i.e. $k \in \{50, \dots, 300\}$.

Esta capa de *embedding* asigna a cada palabra un vector con k valores reales, por lo que es posible representar las N palabras del diccionario con solo k componentes. Para que este proceso sea útil, la representación de cada palabra con k valores debe cumplir la propiedad de que palabras con significado similar tengan valores similares, mientras que palabras con significado diferente tengan valores diferentes. Esto se puede conseguir de dos formas principalmente:

- Crear la matriz de *embedding* de forma no supervisada como preentrenamiento de la red neuronal recurrente.

En este caso, el proceso consiste en utilizar un algoritmo como Word2Vec, que es capaz de extraer relaciones semánticas entre palabras de forma no supervisada observando únicamente los textos disponibles para el problema que intentamos resolver, sin ninguna otra forma de supervisión.

- Crear la matriz de *embedding* a la vez que se entrena la red neuronal recurrente.

En este caso se empieza con una matriz de *embedding* con valores aleatorios y se consideran las entradas de la matriz como parámetros de la red neuronal durante el entrenamiento.

Lectura complementaria

T. Mikolov *et al.*
«Distributed Representations of Words and Phrases and Their Compositionality». Proceedings of the 26th International Conference on Neural Information Processing Systems, Vol. 2, p. 3111-3119, 2013.

El propio proceso de entrenamiento de la red neuronal ajustará los valores de los vectores para que cada palabra quede representada en k dimensiones de tal forma que la respuesta final de la red sea lo mejor posible.

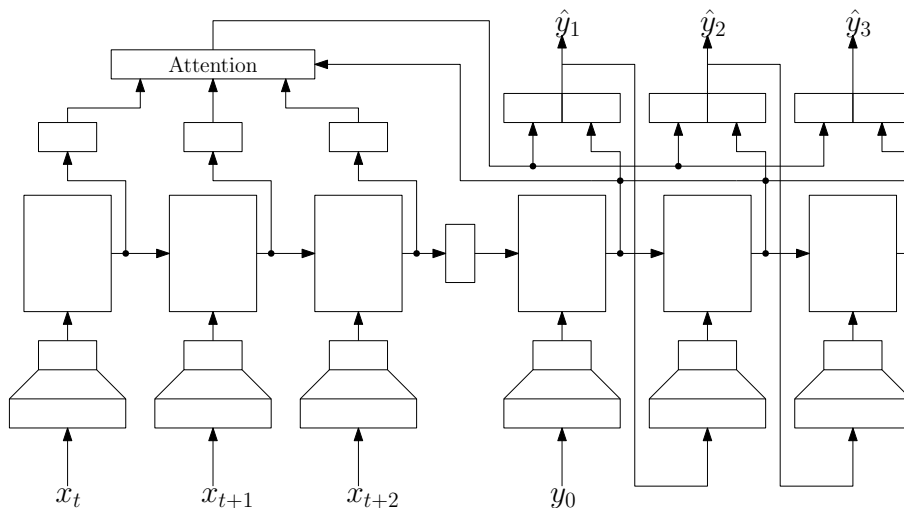
En el caso de optar por una matriz de *embedding* preentrenada es posible utilizar *embeddings* disponibles en Internet que ya están preentrenados con grandes cantidades de textos, como por ejemplo la Wikipedia.

Para escoger entre una forma u otra de crear la matriz de *embedding* deberemos tener en cuenta diferentes puntos, como por ejemplo si disponemos de suficientes textos como para crear un *embedding* de forma no supervisada o si el vocabulario de nuestros textos es demasiado específico como para utilizar un *embedding* genérico obtenido de Internet.

También puede ser interesante mezclar las dos aproximaciones y crear la matriz de *embedding* utilizando primero un preentrenamiento no supervisado y afinando después los valores de los vectores durante el entrenamiento de la red neuronal.

En la figura 19 podemos ver el mismo esquema del mecanismo de atención de la figura 16 pero con la capa de *embedding* aplicada sobre las entradas de la red.

Figura 19. Mecanismo de atención en una red neuronal recurrente con arquitectura *encoder-decoder / sequence-to-sequence*, aplicando primero una capa de *embedding* a la entrada de la red neuronal



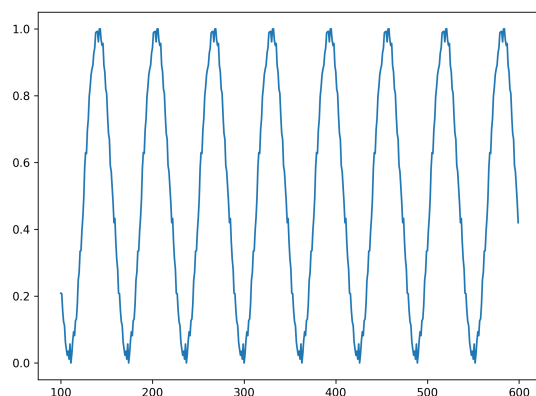
4.2. Ejemplos

A continuación veremos cómo podemos predecir valores futuros de la serie temporal de la figura 18 utilizando su descomposición.

Esta serie temporal ha sido generada sintéticamente, por lo que sabemos que todo aquello que no pertenece a la tendencia o a la estacionalidad es ruido blanco. En este sentido, el siguiente ejemplo muestra como se podrían predecir valores futuros utilizando una red neuronal recurrente en la parte estacionaria de la serie. En un caso con datos reales se podría utilizar un modelo más simple para la parte estacionaria y entrenar una red recurrente para intentar detectar patrones en la componente irregular, pero en el caso de la serie de la figura 18 sabemos que, por construcción, sufriríamos de sobreajuste (estaríamos entrenando un modelo con un conjunto de datos formado completamente por ruido).

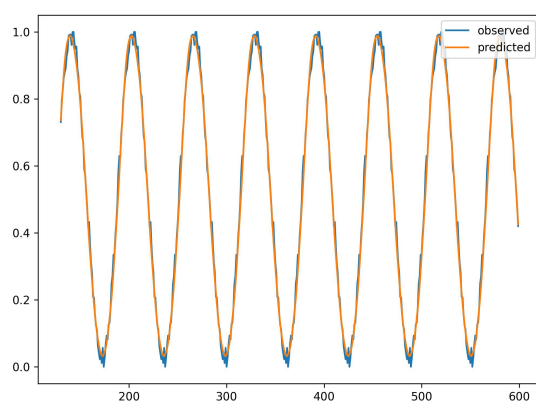
El primer paso del proceso consiste en entrenar una red neuronal recurrente que capture la estacionalidad de los datos. Para ello, es necesario normalizar los valores de la parte estacional de la serie, como podemos ver en la figura 20.

Figura 20. Parte estacional de la serie normalizada



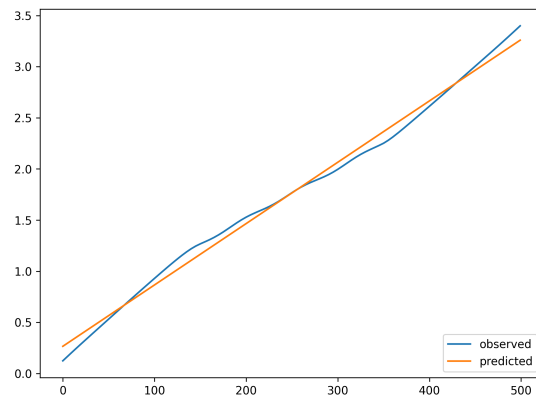
Para predecir la parte estacional de la serie podemos utilizar una red LSTM con 50 unidades en cada celda y que utilice los 30 últimos valores de la serie para predecir el siguiente. Con una serie tan periódica y una red neuronal recurrente se pueden obtener resultados muy buenos, como se muestra en la figura 21.

Figura 21. Predicción de los datos de entrenamiento de la parte estacional de la serie normalizada



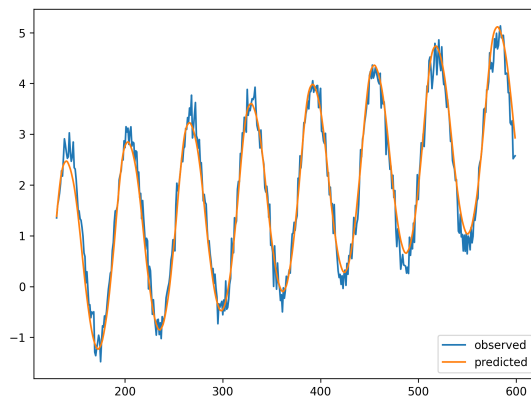
Para predecir la tendencia, podríamos utilizar también una red neuronal, pero es mucho mejor utilizar un modelo más simple para evitar el sobreajuste. En este caso, dado que la tendencia es prácticamente lineal utilizaremos un modelo lineal de regresión simple. En la figura 22 se puede ver el resultado de ajustar la tendencia de los datos de entrenamiento a un modelo lineal.

Figura 22. Predicción de la tendencia de los datos de entrenamiento utilizando un modelo lineal



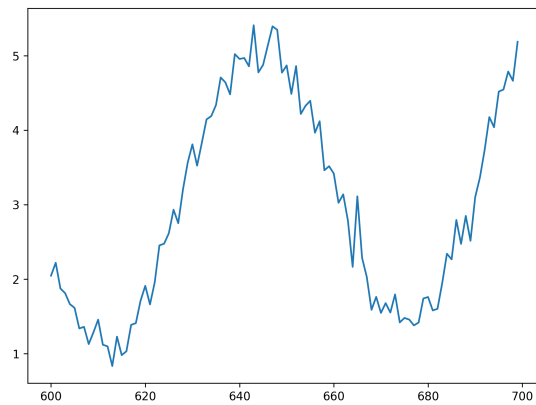
Si sumamos ahora las predicciones del modelo lineal con las predicciones de la red neuronal recurrente obtenemos una predicción para los datos de entrenamiento, que se puede ver en la figura 23.

Figura 23. Suma de las predicciones del modelo lineal y la red neuronal recurrente para los datos de entrenamiento



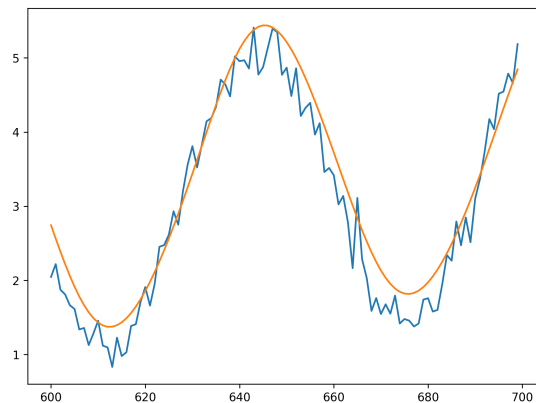
Dado que la red parece haber captado la tendencia y la estacionalidad de los datos podemos utilizarla para la predicción de datos futuros. En la figura 24 podemos ver los siguientes valores de la serie.

Figura 24. Valores de la serie posteriores a los utilizados para el entrenamiento



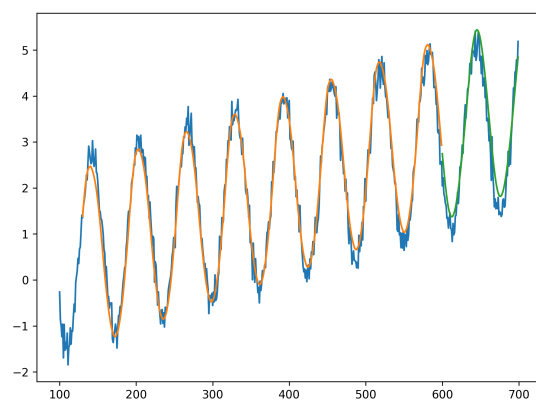
Si aplicamos los modelos ya entrenados para predecir los nuevos datos obtenemos los resultados que muestra la figura 25.

Figura 25. Predicción de los modelos para los datos de test



Por último, podemos ver todos los datos y predicciones de forma conjunta en la figura 26.

Figura 26. Resumen de predicciones para la serie temporal (datos de entrenamiento y de test)



En este caso en particular los datos de la serie temporal han sido generados sintéticamente, por lo que sabemos seguro que la parte residual de la des-

composición está formada principalmente de valores aleatorios. Esto es así en general, por lo que, en principio, no se debe entrenar ningún modelo sobre la parte residual, ya que sino, por definición, sufriríamos de sobreajuste.

Hay casos en cambio, en los que es posible que haya algún patrón escondido en la parte residual de la descomposición, especialmente si los valores de la serie temporal se ven afectados por variables externas que no se están considerando a la hora de hacer la descomposición. En estos casos puede ser interesante entrenar una red neuronal con todas las variables que se consideren convenientes para intentar extraer algo de información de los residuos. Por ejemplo, si la serie temporal representa el consumo de energía de un pueblo turístico, es posible que en la descomposición se detecten los fines de semana o los meses de verano, pero hay festividades como la Semana Santa que cambian de fecha cada año. En este caso, un modelo en el que se indique qué días serán festivos y cuáles no es posible que consiga predecir con sentido parte de los residuos sin caer en el sobreajuste.

Resumen

En este módulo hemos visto los conceptos básicos de las redes neuronales recurrentes. Hemos visto que este tipo de redes es muy útil cuando los datos tienen una estructura secuencial, ya que una red recurrente tratará cada registro del conjunto de datos también de forma secuencial.

Como hemos comentado, en general una red neuronal recurrente podría tener las conexiones recurrentes establecidas de muchas formas diferentes, pero se suelen utilizar celdas para facilitar el trabajo. Los tipos de celdas más utilizados actualmente son las celdas LSTM y las celdas GRU.

El buen funcionamiento de las celdas LSTM y GRU se basa principalmente en dos factores:

- Tienen un canal de memoria al cual no se le aplica en ningún momento una operación que no sea lineal.
- Controlan el flujo de información utilizando funciones sigmoide para simular puertas lógicas.

Dependiendo del tipo de problema, y especialmente si se dispone de gran cantidad de datos, puede ser una buena opción combinar las celdas de una red neuronal recurrente en arquitecturas complejas. Hemos visto en este capítulo la opción de crear redes neuronales recurrentes bidireccionales, profundas o con arquitectura codificador-decodificador. Es posible también unir diferentes tipos de arquitecturas para conseguir redes neuronales con una gran capacidad predictiva.

Por último, hemos visto los fundamentos del mecanismo de atención, el cual es la base de la arquitectura de transformadores que veremos en futuros módulos.

Glosario

atención *m* Mecanismo que permite a una red neuronal con arquitectura seq2seq utilizar información focalizada de la secuencia de entrada según sea el estado de la decodificación.

celda *f* Modelo abstracto para representar la arquitectura de una red neuronal como combinación de operaciones con dos entradas y dos salidas.

GRU *m Gated Recurrent Unit*, tipo de celda utilizada en redes neuronales recurrentes, se puede considerar como una simplificación de una celda LSTM.

LSTM *m Long Short Term Memory*, tipo de celda muy utilizada en redes neuronales recurrentes.

recurrencia *f* Situación en la que la salida de las neuronas de una red neuronal se conecta con la entrada de las neuronas de una capa anterior.

seq2seq *f* Arquitectura de red neuronal con dos partes fundamentales, un codificador y un decodificador, muy utilizada cuando el problema a tratar consiste en la transformación de secuencias de entrada en secuencias de salida (por ejemplo, en problemas de traducción automática).

Bibliografía

Bahdanau, D. et al. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. 3rd International Conference on Learning Representations.

Casas Roma, J., Lozano Bagén, T., Bosch Rué, A. (2020). *Deep learning: Principios y fundamentos*. Barcelona: Editorial UOC.

Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), p. 1724-1734.

Graves, A. et al. (2013). *Speech recognition with deep recurrent neural networks*. International Conference on Acoustics, Speech and Signal Processing, p. 6645-6649.

Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, Vol. 9 (8), p. 1735-1780.

Mikolov, T. et al. (2013). *Distributed Representations of Words and Phrases and Their Compositionality*. Proceedings of the 26th International Conference on Neural Information Processing Systems, Vol. 2, p. 3111-3119.

Schuster, M., Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks*. Trans. Sig. Proc., Vol. 45 (11), p. 2673-2681.

Sutskever, I. et al. (2014). *Sequence to Sequence Learning with Neural Networks*. Proceedings of the 27th International Conference on Neural Information Processing Systems, Vol. 2, p. 3104-3112.